From Python to URL

Written instructions for the Lecture 2 tutorial video (BMI 640)

This document follows the video step by step. You will build a small interactive web application from an empty folder and publish it on the internet. You can type every line yourself, or copy and paste from the code blocks below. Each step ends with the same short routine: test locally, commit, push, and watch the live app update.

All data used here are synthetic. They were generated by a program and contain no real patients.

What you will build

ED Visit Explorer lets a user pick an age group and see the number of emergency department visits, the median wait time, a chart of visits by chief complaint, and a histogram of wait times.

Files you need

- `ed_visits.csv`: the synthetic dataset (2,000 visits). It is in the `data` folder of the GitHub repository for this tutorial: github.com/UmairZ/ed-visit-explorer
- The code for every step is in this document. The finished `app.py` and `requirements.txt` are in the same repository, if you would rather copy than type.

Part 1. Project setup

1.1 Install the tools and create the accounts

What	Where	Notes
Python	python.org/downloads	Windows: check "Add Python to PATH" in the installer.
Git	git-scm.com/downloads	Default options are fine.
Code editor	code.visualstudio.com	Visual Studio Code. Any editor works; VS Code has a built-in terminal.
GitHub account	github.com	Free. Hosts your code online.
Streamlit Community Cloud account	share.streamlit.io	Free. Choose "Continue with GitHub" so it can read your repositories.

Git vs. GitHub. Git is the version control system. It runs on your computer and records changes to your files over time. GitHub is an online platform that hosts Git repositories so a project can be stored and shared.

1.2 Create the project folder and check your tools

1. Create a new folder named `ed-visit-explorer` and open it in VS Code (File, Open Folder).

2. Open the terminal in VS Code (Terminal, New Terminal).
3. Check that Python and Git are installed. Each command should print a version number.

```
python --version
git --version
```

On a Mac, use `python3 --version`.

4. Tell Git who you are. Do this once per computer, using your own name and email.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

1.3 Create and activate a virtual environment

A virtual environment is a private, project-specific set of Python packages, so this project does not interfere with others on your computer.

****	**Windows (PowerShell)**	**Mac / Linux**
Create	`python -m venv .venv`	`python3 -m venv .venv`
Activate	`.venv\Scripts\activate`	`source .venv/bin/activate`

When it is active, `(.venv)` appears at the start of the terminal prompt. If PowerShell says that running scripts is disabled, run this once and then activate again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Next, install Streamlit into the environment:

```
pip install streamlit
```

Part 2. A minimal app, run locally

Create a new file named `app.py` in the project folder (not inside `.venv`) and type:

```
import streamlit as st

st.title("ED Visit Explorer")
st.write("A small app for exploring synthetic emergency department visits.")
```

`import streamlit as st` loads the library and gives it the short name `st`. `st.title` shows a heading and `st.write` shows text. Save the file and run the app from the terminal:

```
streamlit run app.py
```

Your browser opens at a `localhost` address. Only your own computer can see this page, which is the problem the next parts solve. Streamlit reruns the script from top to bottom whenever the page needs to update. Leave this terminal running and open a second terminal (the plus button) for the commands below. Activate the environment in it if `(.venv)` does not appear.

Part 3. Put the project on GitHub

3.1 Two small files

Create `requirements.txt`. It lists the packages the project needs, so another computer (or a server) can rebuild the environment:

```
streamlit
```

Create `.gitignore`. It tells Git which files not to track. The virtual environment is large and specific to your machine, so it stays out of the repository:

```
.venv/
__pycache__/
```

3.2 Create the repository and the first commit

```
git init -b main
git add .
git status
git commit -m "Add minimal app"
```

`git add .` stages your files, `git status` shows what is staged (you should not see `.venv`), and `git commit` saves a snapshot with a message.

3.3 Create the GitHub repository and push

1. On github.com, choose New repository.
2. Name it `ed-visit-explorer` and set it to **Public**. Leave the options for adding a README or .gitignore unchecked.
3. Create the repository and copy its HTTPS address.
4. Connect your local repository to it and push, using your own address:

```
git remote add origin https://github.com/<your-username>/ed-visit-explorer.git
git push -u origin main
```

The first time, a window may ask you to sign in to GitHub. Refresh the repository page to confirm your files are there.

Part 4. Deploy on Streamlit Community Cloud

1. Go to share.streamlit.io and sign in with GitHub.

2. Choose the option to create a new app from an existing GitHub repository. (Button labels change over time; pick the one that deploys from GitHub.)
3. Select your repository `ed-visit-explorer`, the branch `main`, and the main file path `app.py`. Optionally choose a custom web address.
4. Click Deploy and wait a minute or two.

What is happening. The Streamlit server is a computer that knows nothing about your laptop. It copies your repository from GitHub, installs the packages listed in `requirements.txt`, and runs `app.py`. Deployment is reconstructing your execution environment on someone else's computer using the information in your repository. If a package is missing from `requirements.txt`, the server will not have it.

When it finishes you have a public URL. Free apps may go to sleep after a period of inactivity and wake up the next time someone visits.

The routine for every step below

1. Edit the code and save.
2. Check it locally in the browser.
3. If you added a new import, add that package name to `requirements.txt`.
4. Commit and push. The deployed app updates a minute or so later.

```
git add .
git commit -m "Describe the change"
git push
```

Part 5. Add the data and summary numbers

Create a folder named `data` inside the project and put `ed_visits.csv` in it. It is a CSV file: one row per visit, with columns for visit ID, arrival time, age, age group, sex, chief complaint, acuity level, arrival mode, and wait in minutes.

Replace `app.py` with the version below. It adds `pandas`, which is a library for working with tables of data, loads the CSV, and shows two numbers and the table.

```
import pandas as pd
import streamlit as st

st.title("ED Visit Explorer")
st.write("A small app for exploring synthetic emergency department visits.")

visits = pd.read_csv("data/ed_visits.csv")

st.metric("Number of visits", len(visits))
st.metric("Median wait (minutes)", visits["wait_minutes"].median())
st.dataframe(visits)
```

Add `pandas` to `requirements.txt` because the app now imports it:

```
streamlit
```

```
pandas
```

Commit and push with the message "Add data and summary metrics". The data file is part of the repository, so it reaches the server too.

Part 6. Add the age group filter

`st.selectbox` creates a dropdown. The chosen value is stored in `choice`, and every time the user changes it, Streamlit reruns the script. An `if` statement then chooses which rows to keep, and `st.columns(2)` places the two metrics side by side. Everything below the filter now uses `selected` instead of `visits`.

```
import pandas as pd
import streamlit as st

st.title("ED Visit Explorer")
st.write("A small app for exploring synthetic emergency department visits.")

visits = pd.read_csv("data/ed_visits.csv")

age_groups = ["All ages"] + sorted(visits["age_group"].unique())
choice = st.selectbox("Age group", age_groups)

if choice == "All ages":
    selected = visits
else:
    selected = visits[visits["age_group"] == choice]

col1, col2 = st.columns(2)
col1.metric("Number of visits", len(selected))
col2.metric("Median wait (minutes)", selected["wait_minutes"].median())

st.dataframe(selected)
```

No new package, so `requirements.txt` does not change. Commit and push with the message "Add age group filter".

Part 7. Add a chart of visits by chief complaint

Add a subheading and Streamlit's built-in bar chart between the metrics and the table. `value_counts()` counts how many times each chief complaint appears in the selected rows.

```
import pandas as pd
import streamlit as st

st.title("ED Visit Explorer")
st.write("A small app for exploring synthetic emergency department visits.")

visits = pd.read_csv("data/ed_visits.csv")
```

```
age_groups = ["All ages"] + sorted(visits["age_group"].unique())
choice = st.selectbox("Age group", age_groups)

if choice == "All ages":
    selected = visits
else:
    selected = visits[visits["age_group"] == choice]

col1, col2 = st.columns(2)
col1.metric("Number of visits", len(selected))
col2.metric("Median wait (minutes)", selected["wait_minutes"].median())

st.subheader("Visits by chief complaint")
st.bar_chart(selected["chief_complaint"].value_counts())

st.dataframe(selected)
```

Commit and push with the message "Add complaint chart".

Part 8. Add a histogram and finish

The median is one number, but waits vary. A histogram shows the spread. This step imports a charting library named Altair at the top of the file, so add `altair` to `requirements.txt`. It also puts the table in a collapsible section and adds a caption.

```
import altair as alt
import pandas as pd
import streamlit as st

st.title("ED Visit Explorer")
st.write("A small app for exploring synthetic emergency department visits.")

visits = pd.read_csv("data/ed_visits.csv")

age_groups = ["All ages"] + sorted(visits["age_group"].unique())
choice = st.selectbox("Age group", age_groups)

if choice == "All ages":
    selected = visits
else:
    selected = visits[visits["age_group"] == choice]

col1, col2 = st.columns(2)
col1.metric("Number of visits", len(selected))
col2.metric("Median wait (minutes)", selected["wait_minutes"].median())

st.subheader("Visits by chief complaint")
st.bar_chart(selected["chief_complaint"].value_counts())

st.subheader("Distribution of wait times")
histogram = (
    alt.Chart(selected)
```

```
    .mark_bar()
    .encode(
        alt.X("wait_minutes:Q", bin=alt.Bin(step=10), title="Wait (minutes)"),
        alt.Y("count()", title="Number of visits"),
    )
)
st.altair_chart(histogram, width="stretch")

with st.expander("Show the data"):
    st.dataframe(selected)

st.caption("All data are synthetic. No real patient information is used.")
```

Final `requirements.txt`:

```
streamlit
pandas
altair
```

Commit and push with the message "Add wait-time histogram". Open your live URL and try the dropdown.

Where to go from here

- **Look at your history.** On your GitHub repository, open the commit list. Each step of this tutorial is a saved version you could return to.
- **Extend the app.** Try a filter for acuity level, or a chart of median wait by chief complaint, using the same routine: edit, test, commit, push.
- **Remember what this is.** It is a prototype using synthetic data. A working URL does not mean an application is ready for clinical use. With real patient information you would need to consider access control, security, privacy, and your institution's policies before putting anything online, and a public repository on a free hosting service would not be appropriate. Never put passwords or keys in a public repository.

Common problems

Message or symptom	Usual cause and fix
`streamlit` is not recognized, or `ModuleNotFoundError`	The virtual environment is not active, or the package is not installed. Activate it (look for `(.venv)`) and run `pip install streamlit pandas altair`.
`FileNotFoundError: data/ed_visits.csv`	Run `streamlit run app.py` from the project folder, and check that the file is at `data/ed_visits.csv`.
Works on your computer but the live app shows a `ModuleNotFoundError`	A package is missing from `requirements.txt`. Add it, commit, and push.

Message or symptom	Usual cause and fix
Live app did not change after a push	Check that `git push` finished without an error and that you committed the file you edited. Wait a minute, then refresh. The Manage app menu on Streamlit shows the logs.
Live app asks to be woken up	Free apps sleep after inactivity. Click the button to restart it.